

Intermediate Docker Commands

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

Docker refresher

- Docker is a container runtime
- Is designed to run and manage various containerized applications on Windows, Mac, and Linux
- Can run containers using pre-built images, or create our own
- Dockerfiles are text files used to Docker container images
- Containers are instances of a given Docker image

The DataCamp [Introduction to Docker course](#) is a pre-requisite to this course

A screenshot of a course card for 'Introduction to Docker' on DataCamp. The card has a dark blue background. At the top, it says 'INTERACTIVE COURSE' in small white letters. Below that, the title 'Introduction to Docker' is in large white letters. There are two buttons: a green 'Continue Course' button and a white 'Bookmark' button with a yellow bookmark icon. At the bottom, there are four tags: 'Intermediate' with two green dots, '4 hours' with a clock icon, '15 videos' with a play icon, and '56 exercises' with a code icon.

INTERACTIVE COURSE

Introduction to Docker

[Continue Course](#) [Bookmark](#)

●● Intermediate ⌚ 4 hours ▶ 15 videos <> 56 exercises

Docker commands

- `docker run`
- `docker stop`
- `docker build`
- `docker --help`
 - Provides a list of potential Docker commands
- `docker COMMAND --help`
 - `docker run --help`
 - Provides options for the `docker run` command

Temporary containers

- Docker containers are usually created with `docker run`
- Containers remain even after stopping / exiting
- Often want to run a container instance and remove it immediately upon exit
 - Development
 - Testing
 - Scripts
- `docker run --rm`
 - `docker run --rm alpine:latest`
 - `/bin/sh`
- Referenced as 'clean-up' or 'remove'

docker ps

- Used for determining name, id, status, and other attributes of containers on a given machine running Docker
- Use the `-a` flag to get more information about existing containers
 - `docker ps -a`
- Will cover how to get extremely detailed information about containers later in the course

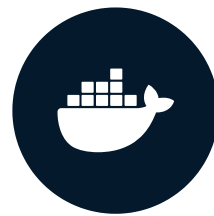
```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
588bb3a3134e	nginx	"/docker-entrypoint..."	49 seconds ago	Up 47 seconds
8a0578a9b33c	ubuntu	"/bin/bash"	About a minute ago	Exited (0) About a minute ago

Let's practice!
INTERMEDIATE DOCKER

Mounting the host filesystem

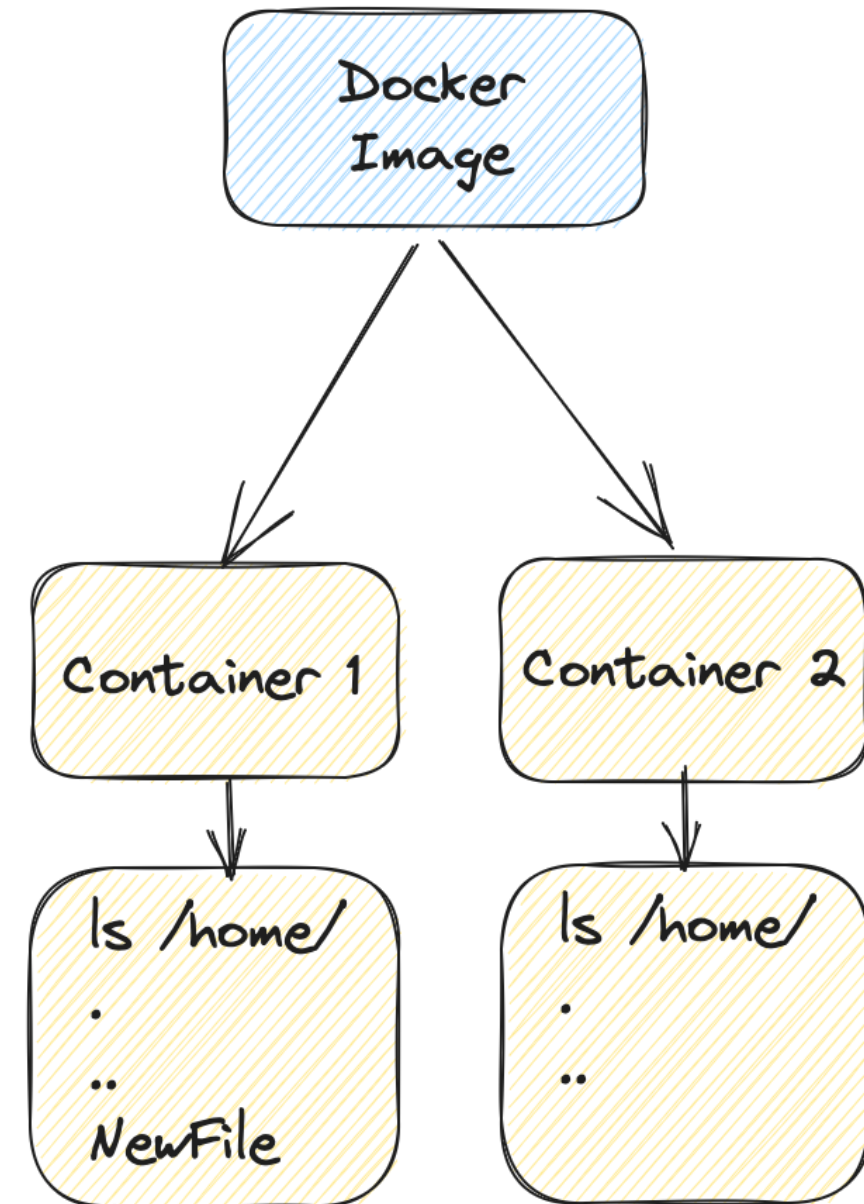
INTERMEDIATE DOCKER



Mike Metzger
Data Engineer

Container filesystems

- Container instances each have their own filesystem
 - Based off the image the container was created with
- Any changes are tied to that specific container instance
- Any changes are maintained across restarts
 - For that instance **only**
- New containers only have the data in the image, not instance specific changes



Sharing files or directories

- Can attach specific files or directories to containers
- Allows for persistence of data, without maintaining a specific container
- Can upgrade container to new version but safely keep data / changes
- Known as **bind-mount**
- Can be read-only or read/write
- Note: When files or directories are attached to a container, they are not accessible to the host until the container is shutdown

Using the -v option

- bind-mounts most often use the `-v` flag
- `-v <source>:<destination>`
- Multiple `-v` commands permitted
- Can also use the `--mount` option
- Note: bind-mount **hides** any content already present in the destination directory

```
docker run -v ~/html:/var/www/html \  
    nginx
```

```
docker run \  
    -v ~/pgdata:/opt/data \  
    -v ~/pg.conf:/etc/pg.conf \  
    postgresql
```

Let's practice!
INTERMEDIATE DOCKER

Persistent volumes

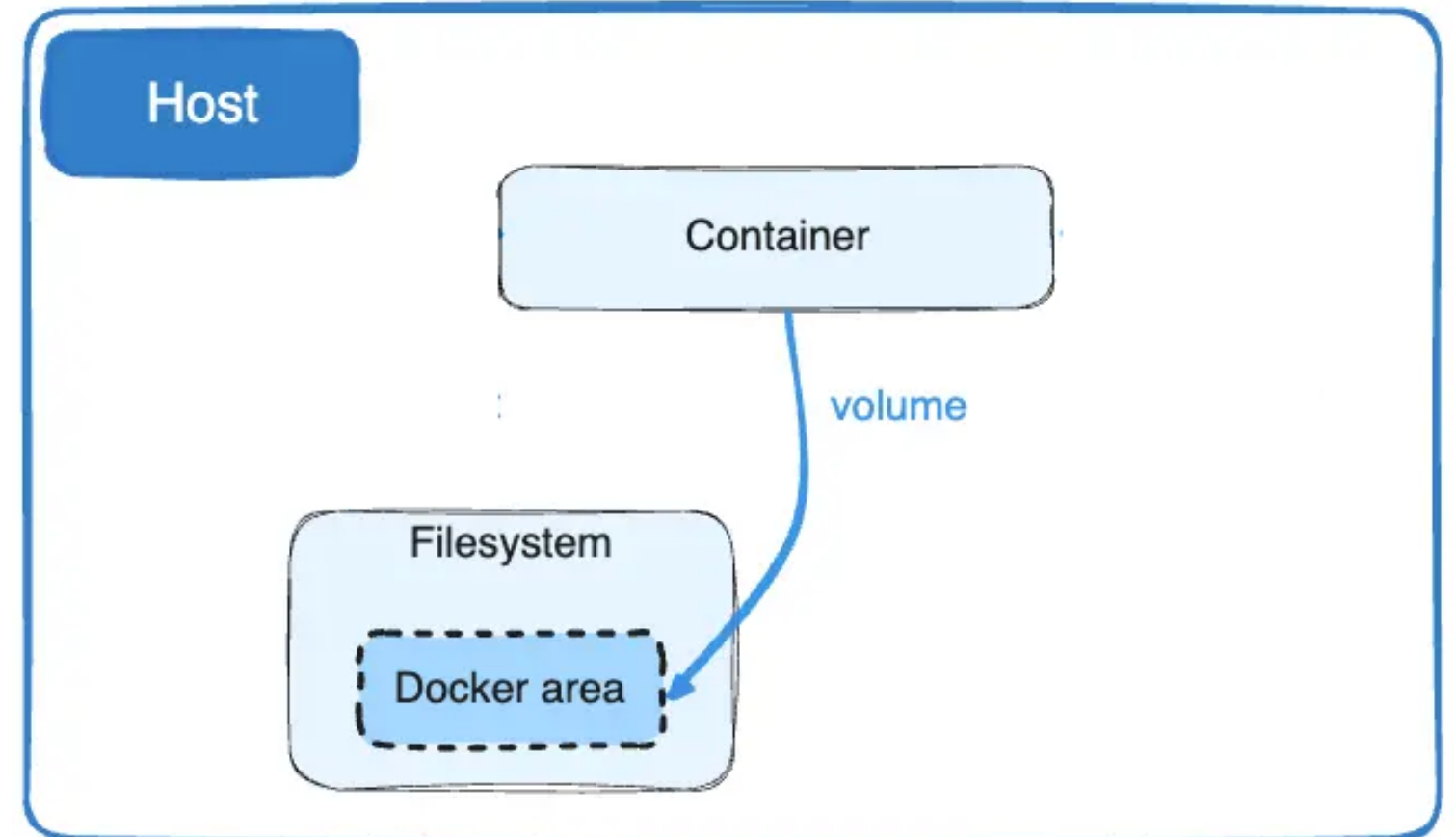
INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

What is a volume?

- **Volumes** are an option to store data in Docker, unrelated to the container image or host filesystem
- Are managed from the command line (or API)
- Can share with multiple containers
- Higher performance than file share / bind mounts
- Exist until removed



¹ Image modified from <https://docs.docker.com>

Managing volumes

- `docker volume`
- `docker volume create <volumename>`
- `docker volume ls` or
`docker volume list`
- `docker volume inspect`
 - Provides assorted metadata about the volume, including Name, Mountpoint, Options, and so forth
- `docker volume rm`

Volume creation example

```
bash> docker volume create sqldata
```

```
sqldata
```

```
bash> docker volume ls
```

DRIVER	VOLUME NAME
local	2f2b7f710551e004dcdd9edf4cad31c37826b428de12f1c04ca02305d216ab00
local	14da7ff0c6eb29f644e6f9f9d59bbcf56b3699c04881dd7cbcaa9ecd6bef239c
local	150aa3c5c7aee30ffd1ec7ecf39f03989bf561536a9413ebed96ffbaa537d103
local	sqldata
...	

Volume inspect example

```
bash> volume inspect sqldata
[
  {
    "CreatedAt": "2024-01-27T04:27:51Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/sqldata/_data",
    "Name": "sqldata",
    "Options": null,
    "Scope": "local"
  }
]
```


Attaching volumes

- Uses the `-v` command
 - `docker run -v <volumename>:
<destination path>:<options>`
 - Volume name is name of existing volume
 - Destination path is the location the volume will be mounted (such as `/data`)
 - Options are optional comma-separated list of values such as `ro` for read-only.
- `--mount` exists as with bind-mounts

```
$ docker run -v sqldata:/data postgres
```

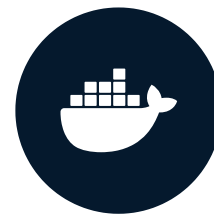
Drivers

- Methods of storing Docker volumes
- Can include:
 - Local filesystem (default)
 - NFS (Unix filesharing)
 - SMB / CIFS (Windows filesharing)

Let's practice!
INTERMEDIATE DOCKER

Networking refresher

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

What is networking?

- A computer network consists of systems communicating via a defined method
- Varying levels of communication, physical or logical, referred to as *protocols*
- Common physical networks include Ethernet and WiFi
- Logical networking includes TCP/IP, HTTP, and SMTP
- Networks are defined in various layers or levels. Often referred to as a networking stack



¹ Photo by Jordan Harrison on Unsplash.

Networking terms

- Host
 - General term for a computer
- Network
 - Group of hosts
- Interface
 - Actual connection from a host to a network, such as Ethernet or WiFi
 - Can be virtual, meaning entirely in software
- LAN
 - Local Area Network, or set of computers at a given location
- VLAN
 - Virtual LAN, or a software LAN

Internet Protocol

- IP
 - Internet protocol, method to connect between networks using IP addresses
- IPv4
 - Version of IP supporting 4.2 billion addresses, currently exhausted
- IPv6
 - Newer version of IP, supporting 2^{128} addresses, still being deployed

IPv4: 10.10.10.1

IPv6: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

TCP / UDP

- TCP
 - Transmission Control Protocol, used to reliably communicate between hosts on IP networks
- UDP
 - User Datagram Protocol, used to communicate between hosts on IP where communication is not required.

Ports

- Port
 - Addresses services on a given host, a value between 0 and 65535, used to communicate between hosts via TCP or UDP
 - Ports below 1024 are typically reserved for privileged accounts
 - Values above 1024 are usually ephemeral or temporary ports
 - Applications **listen** on a port.

Application protocols

- HTTP/HTTPS
 - Application protocol, defaulting to TCP port 80 for web communication. Secure version on TCP 443
- SMTP
 - Email transfer protocol, over TCP port 25
- SNMP
 - Network management protocol, over UDP port 161

Docker and networking

- Can communicate between containers
- Can communicate with the host system
- Depending on settings can communicate with external hosts
- Typical communication is handled by exposing ports from container to host
- Acts like a translation between containers and hosts

Docker and IP

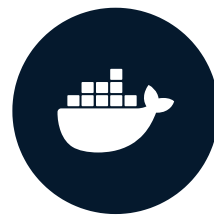
- Containers can have IP addresses
- Use `ifconfig <interface>` or `ip addr show <interface>` from within container to find addresses
- Use `ping -c <x> <host>` to verify connectivity
 - `ping -c 3 myhost`

```
repl@4221eef4-d3b6-45ef-b95a-71e78ac62619:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 9001
    inet 10.4.77.205  netmask 255.255.255.255  broadcast 0.0.0.0
    inet6 fe80::5457:73ff:fe94:f6e4  prefixlen 64  scopeid 0x20<link>
    ether 56:57:73:94:f6:e4  txqueuelen 0  (Ethernet)
    RX packets 1811  bytes 272039 (272.0 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 1295  bytes 137826 (137.8 KB)
    TX errors 0  dropped 1 overruns 0  carrier 0  collisions 0
```

Let's practice!
INTERMEDIATE DOCKER

Making network services available in Docker

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

Network services

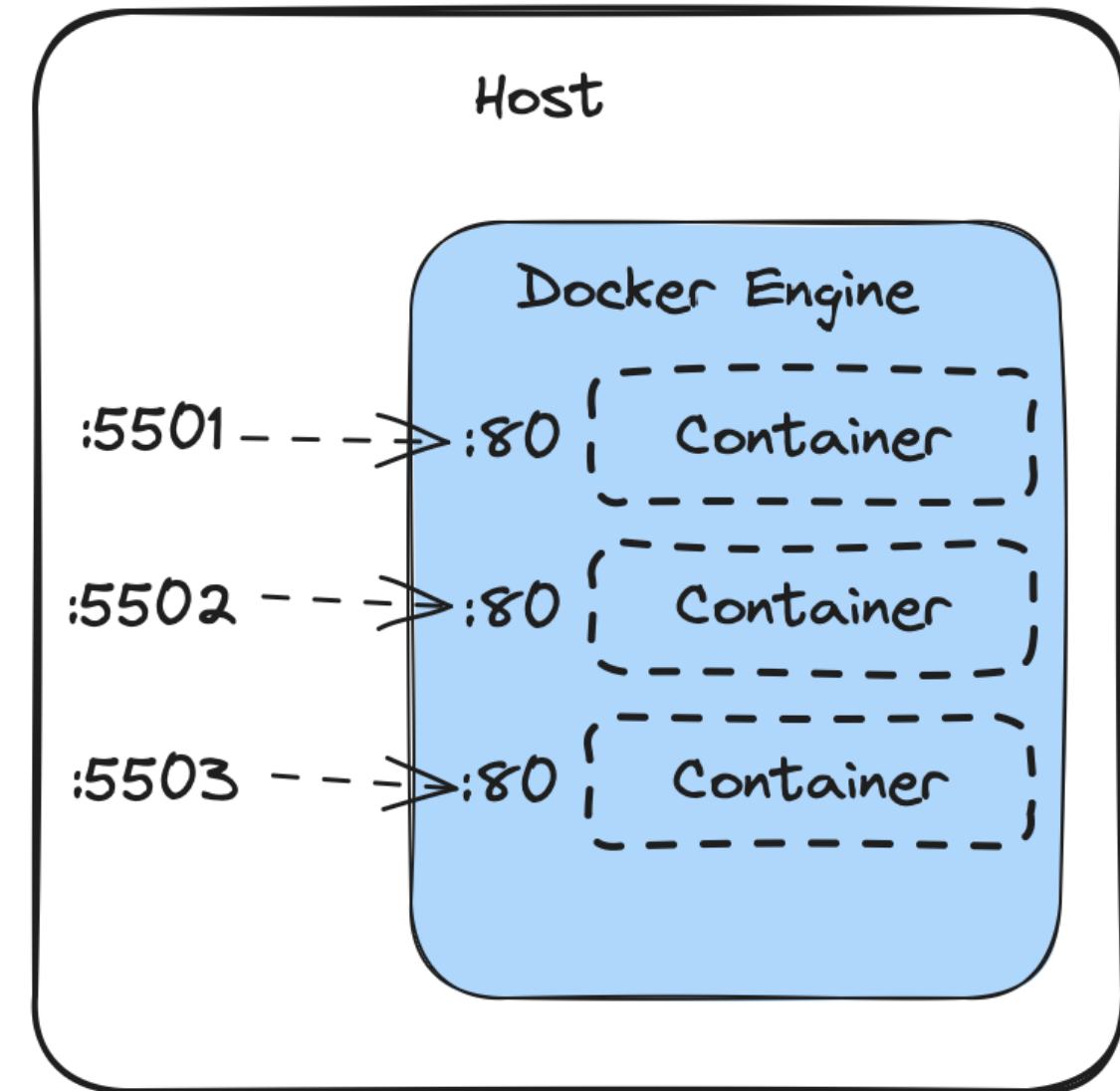
- Network services **listen** on a given port
- Only one program can listen on an IP:port combo at a given time
 - For example, 10.1.2.3:80 would be listening on 10.1.2.3 on port 80.
- Consider trying to debug different versions of a web server that listens on port 80
 - Could only run one copy of the application at a time given that it listens only on that port

Containerized services

- Wrapping application in a container means that each container can now listen on that port (as the IP:port combo is different, each container has a different IP)
- Can have multiple copies of the containers running at once
- But how to connect to container's version of application from the host?

Port mapping

- The answer is the use of **port mapping**, or port forwarding / translation
- Port mapping takes a connection to a given IP:port and automatically forwards it to another IP:port combo
- In this case, we could map an unused port on our host and point it to port 80 on the container(s)
- The Docker engine can handle this automatically if we configure it to



Enabling port mapping

- To enable port mapping on a given container, we use the `docker run` command, and the `-p` flag
- `-p <host port>:<container port>`
- `-p 5501:80`
- Can have multiple `-p` flags for different ports

```
repl@host:~$ docker run -p 5501:80 nginx
```

```
repl@host:~$ docker ps -a
```

CONTAINER ID	IMAGE	... PORTS	NAMES
84266724ff47	nginx	... 0.0.0.0:5501->80/tcp, :::5501->80/tcp	coiled_elgama1

Let's practice!
INTERMEDIATE DOCKER

Exposing ports with Dockerfiles

INTERMEDIATE DOCKER



Mike Metzger
Data Engineer

Exposing services

- `EXPOSE` command
- Defines which ports the container will use at runtime
- Can be defined as `<number>` , `<number>/tcp` , or `<number>/udp`
 - Such as `EXPOSE 80` or `EXPOSE 80/tcp`
- Multiple entries permitted
- Used as a documentation method

Using the `-p` / `-P` flags

- Still requires use of `-p` or `-P` options to `docker run` to make the ports available outside the container
- The `-P` option will automatically map an ephemeral port to the exposed port(s). Must use `docker ps -a` to see which ports are mapped.
- Using `-p<host port>:<container port>` allows use of specific ports.

EXPOSE example

```
# Dockerfile
FROM python:3.11-slim
ENTRYPOINT ["python", "-mhttp.server"]
# Let the Docker engine know
# port 8000 should be available
EXPOSE 8000
```

- Create a container from the image

```
docker run pyserver
```

- Print the state of the container

```
docker ps -a
```

CONTAINER ID	IMAGE	...	PORTS	NAMES
8c3d320255ae	pyserver	...	8000/tcp	angry_chaum

Making ports reachable

- Automatically map temporary port from host to the container

```
docker run -P pyserver
```

```
docker ps -a
```

CONTAINER ID	IMAGE	...	PORTS	NAMES
6bb458ef25da	pyserver	...	0.0.0.0:55001->8000/tcp	beautiful_lamarr

Finding exposed ports

- `docker inspect` provides a lot of information

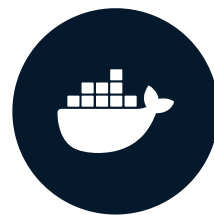
```
docker inspect <id>
```

```
"NetworkSettings": {  
  "Bridge": "",  
  "Ports": {  
    "8000/tcp": [{  
      "HostIp": "0.0.0.0",  
      "HostPort": "55001"  
    }]  
  },  
  ...  
}
```

Let's practice!
INTERMEDIATE DOCKER

Docker networks

INTERMEDIATE DOCKER



Mike Metzger

Data Engineering Consultant

Docker networking

- Docker has extensive networking options
- Can create networks to communicate between containers, host, and external systems
- Will cover various commands to interact with networks.

Docker networking types

- Docker supports different networking types, using drivers
 - **bridge**: Default driver, allows connections out, connections in if exposed
 - **host**: Allows full communication between host and containers
 - **none**: Isolate container from network communications
 - Many others, including custom drivers
- Will mostly use the bridge driver to create our own networks

Working with Docker networks

Several commands:

- `docker network`
 - `docker network <command>`
 - `docker network <command> --help`
 - `docker network ls` to list all docker networks on the host
 - `docker network create` to create a network
 - `docker network rm` to remove a network

Docker network example

- Create a Docker network named `mynetwork`

```
rep1@host:~$ docker network create mynetwork
```

```
5ff0febab98f73b74dd753eb44a30f7d7291052b3b1d58b0134589221cb8e33d
```

```
rep1@host:~$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
2edc5ae4838c	bridge	bridge	local
a92988382711	host	host	local
5ff0febab98f	mynetwork	bridge	local
5464ed866dad	none	null	local

Attaching containers to networks

- How to connect container to a network?
- `docker run --network <networkname> ...`
- `docker run --network mynetwork ubuntu bash`
- Can also connect containers later
 - `docker network connect <networkname> <container>`
 - `docker network connect mynetwork ubuntu-B`

docker network inspect

- How to check details of network?
- `docker network inspect <networkname>`
- Provides configuration info and IP addresses assigned to containers

```
repl@host:~$ docker network inspect mynetwork
```

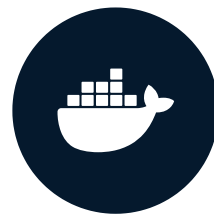
docker network inspect example

```
"Name": "mynetwork",  
...  
"Driver": "bridge",  
...  
Containers": {  
    "2be08aa942029191350d4bceb8816254af8713dd6f7dcbadcab8f068f"  
        "Name": "unruffled_kare",  
        "EndpointID": "29739356ae200e1e901d2eabef05efaca0fb37e1a4e1a4c3bf369",  
        "MacAddress": "02:42:ac:12:00:02",  
        "IPv4Address": "172.18.0.2/16",  
        "IPv6Address": ""  
    }  
}
```

Let's practice!
INTERMEDIATE DOCKER

Optimizing Docker images

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

Docker image explanation

- Docker images are the base of a given container
- Holds all content initially available to a container instance

Docker image concerns

- Tempting to add all potentially needed components to an image
- Size becomes large / unwieldy
- Difficult to handle security / updates due to dependency issues
- Harder to combine containers without wasting space / bandwidth

Docker image recommendations

- Split containers to the smallest level needed
- Easier to combine multiple containers later vs. building a single large image
- Like
 - building with reusable components
 - vs. building from scratch each time
- Updates to specific software only affect containers using that image instead of all containers needing the update
- Can optimize for size, making use and distribution much easier

Docker image breakdown example

- Consider a data engineering project using the following software:
 - Postgresql database
 - Python ETL software
 - Web server software
- Possible to use a single image, but we would need to update the image each time we had an update to the ETL or web server setup.
- What would happen if we needed to add another web server?

```
FROM ubuntu
RUN apt update
RUN apt install -y postgresql
RUN apt install -y nginx
RUN apt install -y python3.9
...
```


Example with minimized containers

- Better options with Docker
- Split each into its own container
 - Postgresql database container
 - Python ETL components
 - Web server
- Can build an optimized configuration for our use, and can add / remove components as needed

```
bash> docker run -d postgresql:latest
bash> docker run -d nginx:latest
...
```

Determining image size

- Using `docker images`
- Shows individual image details, including size
- More in-depth options covered later

```
bash> docker images
```

REPOSITORY	TAG	SIZE
postgres	latest	448MB
postgres	15	442MB
apache/airflow	2.7.1-python3.9	1.4GB
alpine	latest	7.73MB

Let's practice!
INTERMEDIATE DOCKER

Understanding layers

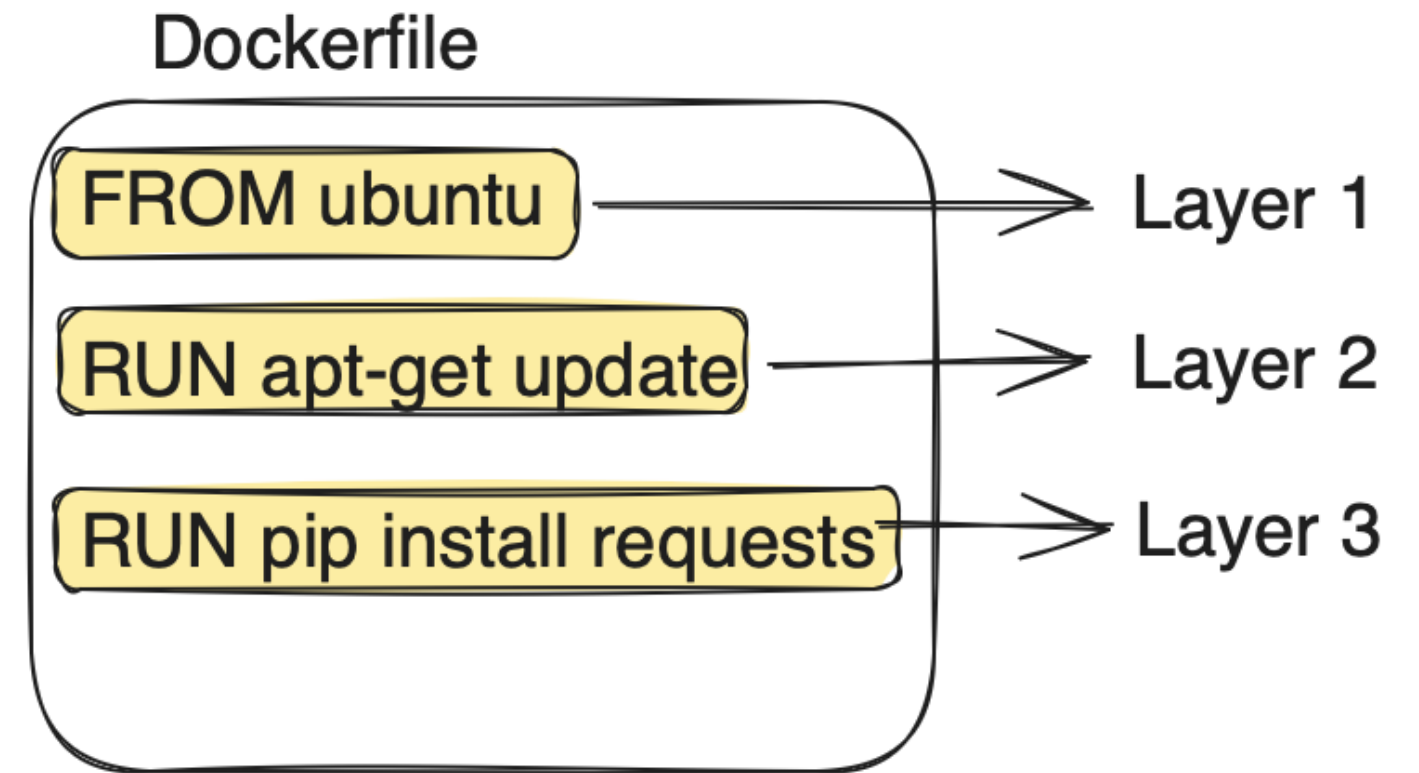
INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

Docker layers

- Docker images are made up of layers
- A layer generally references a change or command within a Dockerfile
- Layers can be cached / reused
- The order of commands within a Dockerfile can affect whether layers are reused



Why do we care about layers?

- Reusability
 - Faster build time
 - Smaller builds

docker image inspect

- How to determine the layers within an image?
- `docker image inspect <img id | name>`
- Provides much information about the content of a Docker image
- The `RootFS:Layers` section provides details about layers in a given Docker image

```
repl@host:~$ docker image inspect alpine
[
  {
    "Id": "sha256:05455a08881ea9cf0e752bc48e61bbd71a34c029bb13df01e40e3e70e",
    "RepoTags": [
      "alpine:latest"
    ],
    "Created": "2024-01-27T00:30:48.743965523Z",
```


docker image inspect example

```
bash> docker image inspect postgres:latest
```

```
"RootFS": {  
  "Type": "layers",  
  "Layers": [  
    "sha256:6f2d01c02c30cc1ffac781aff795cba8eeb29cc27756fe37bf525169856369c6",  
    "sha256:c6ad2d5a3cad837ae66b5560e9c577bfad062556b1f00791d8d733ce44a577ce",  
    "sha256:2153552a84ccb7e4a28a50e766b72345072e59f8af0ff068baf98b413132e0c",  
    "sha256:6c00217b1e4b15c25eb3f6e28b1af8c295f469568014621e31a4c5eb5a8aca6f",  
    "sha256:167177d78e2a33aa822faebe9f01683c648ae78179059db05cd25737f215c305",  
    ...  
  ]  
}
```


jq command-line tool

- Sometimes difficult to analyze the results from `docker image inspect`
- `jq` commandline tool is used to read JSON data, like what's returned from `docker image inspect`
- Can use `jq` to query data

jq recipes with Docker

- Method to see just a specific section, for example the `RootFS` data:
 - `docker image inspect <id> | jq '[0] | .RootFS'`

```
{  
  "Type": "layers",  
  "Layers": [ "sha256:0f5c115c5eea96...",  
              "sha256:20792593831cdc..."  
            ]  
}
```

jq recipes with Docker (part 2)

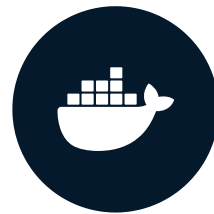
- Method to count number of layers using `jq` :
 - `docker image inspect <id> | jq '[0] | {LayerCount: .RootFS.Layers | length}]'`

```
{  
  "LayerCount": 2  
}
```

Let's practice!
INTERMEDIATE DOCKER

Multi-stage builds

INTERMEDIATE DOCKER



Mike Metzger

Data Engineering Consultant

Single-stage builds

- Typical Docker images are created using a single `FROM` command
- Each addition to the source image adds space and makes its management
- Consider an application that must be compiled prior to use
 - You can add all the necessary components to the image, compile it, and then configure the final image for use
 - This often leaves superfluous content in the image even if it is not used

```
FROM ubuntu
RUN apt update
RUN apt install gcc -y
...
RUN make
CMD ["data_app"]
```

Multi-stage builds

- Multi-stage builds use multiple containers
- Typically has one or more build stages
- Final components are copied into a final container image
- The build stages are then removed automatically
 - Saving space and minimizing the size of the container image
- Uses some additional syntax in the Dockerfile
 - `AS <alias>`
 - `COPY --from=<alias>`

Multi-stage build example

```
# Create initial build stage
FROM ubuntu AS stage1

# Install compiler and compile code
RUN apt install gcc -y

...

RUN make


# Start new stage to create final image
FROM alpine-base

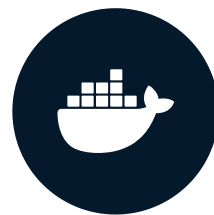
# Copy from first stage to final
COPY --from=stage1 /data_app /data_app

# Run application on container start
CMD ["data_app"]
```


Let's practice!
INTERMEDIATE DOCKER

Multi-platform builds

INTERMEDIATE DOCKER

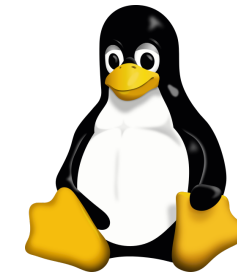


Mike Metzger

Data Engineering Consultant

Multi-platform?

- What does multi-platform mean?
 - Different OS types
 - `linux`
 - `windows`
 - `macos`
 - Different CPU types
 - `x64_64` or `amd64`
 - `arm64`
 - `arm7`
- Usually referred to as `os/cpu` , such as `linux/amd64`



arm

Creating multi-platform builds

- Is built on multi-stage build behavior
- The initial / build stage tends to use cross-compilers and relies on the architecture of the host system
- Final stage uses the architecture / OS for the intended target.

Multi-platform Dockerfile options

- Build stage uses the `--platform=$BUILDPLATFORM` flag
 - `$BUILDPLATFORM` represents the platform of the host running the build
- Sometimes uses the `ARG` directive
 - Passes local environment variables into the Docker build system
 - In this case, `TARGETOS` and `TARGETARCH`
 - `ARG TARGETOS TARGETARCH`
 - The environment variables at the host level can be defined previously or using the `env` command.

Multi-platform example

```
# Initial stage, using local platform
FROM --platform=$BUILDPLATFORM golang:1.21 AS build

# Copy source into place
WORKDIR /src
COPY . .

# Pull the environment variables from the host
ARG TARGETOS TARGETARCH

# Compile code using the ARG variables
RUN env GOOS=$TARGETOS GOARCH=$TARGETARCH go build -o /final/app .

# Create container and load the cross-compiled code
FROM alpine
COPY --from=build /final/app /bin
```

Building a multi-platform build

- To create a multi-platform build, instead of using `docker build`, we must use `docker buildx` with assorted options
- `docker buildx` provides more commands and capabilities over `docker build`, including the option to specify a platform

```
docker buildx build --platform linux/amd64,linux/arm64 -t multi-platform-app .
```

- Prior to running the build, we must also have a new *builder* container present. This is done with the `docker buildx create --bootstrap --use` command.

Let's practice!
INTERMEDIATE DOCKER

Introduction to Docker Compose

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

What is Docker Compose?

- Additional command-line tool for Docker
- Define and manage multi-container applications
- Specify containers, networking, and storage volumes in a single file
 - `compose.yml` or `compose.yaml`
 - Older compose files may be named `docker-compose.yaml`
- Easy to share / demo applications

Example compose.yaml

```
# Define the services
services:
  # Define the container(s), by name
  webapp:
    image: "webapp"
    # Optionally, define the port forwarding
    ports:
      - "8000:5000"
  # Define any other containers required
  redis:
    image: "redis:alpine"
```

Starting an application

- `docker compose up`
 - On older systems, `docker-compose up`
 - `docker compose -f <yaml> up`
 - `docker compose up -d`

```
$ docker compose up
[+] Running 2/0
✓ Network composetest_default      Created
✓ Container composetest-redis-1    Created
✓ Container composetest-web-1      Created
Attaching to redis-1, web-1
redis-1 | 1:C 11 Mar 2024 04:09:51.754 * o000o000o000o Redis is starting o000o000o000o
web-1   | * Serving Flask app 'app.py'
web-1   | * Running on http://127.0.0.1:5000
```

Checking status of applications

- `docker compose ls`

```
$ docker compose ls
```

NAME	STATUS	CONFIG FILES
webapp	running (2)	/webapp/docker-compose.yml

Stopping an application

- `docker compose down`
 - `docker-compose down`
 - `docker compose -f <yaml> down`

```
$ docker compose down
```

```
[+] Running 3/3
```

- ✓ Container composetest-redis-1 Removed
- ✓ Container composetest-web-1 Removed
- ✓ Network composetest_default Removed

Let's practice!
INTERMEDIATE DOCKER

Creating compose.yaml files

INTERMEDIATE DOCKER



Mike Metzger

Data Engineering Consultant

YAML

- Yet Another Markup Language
 - YAML Ain't Markup Language
- Text file, but spacing matters (like Python)
- Used in many development scenarios for configuration
- Rules can be tricky, mainly keep entries lined up as in examples

```
services:
  postgres:
    container_name: postgres
    image: postgres:latest
    ports:
      - "5432:5432"
    restart: always
  pgadmin:
    container_name: pgadmin
    image: dpage/pgadmin4:latest
    ports:
      - "5050:80"
    restart: always
```

Main sections

- Different sections handle different components
- `services:` list the containers to load
- `networks:` handles networking definitions
- `volumes:` controls any volume mounting
- `configs:` handles configuration options without custom images
- `secrets:` Provides options to handle passwords, tokens, API keys, etc
- Refer to the Docker Compose

[Documentation](https://docs.docker.com/compose/compose-file/) for more information

```
services:
  ... # Define containers
networks:
  ... # Define any networking details
volumes:
  ... # Define storage requirements
configs:
  ... # Define special config details
secrets:
  ... # Define passwords / etc
```

¹ <https://docs.docker.com/compose/compose-file/>

Services section

- Defines all required resources for the application
- Primarily specifies the containers and images to be used
- Extensive options available, but only apply to the individual container(s)
- Indentation is applied as needed
- First subsection is the name of each component, followed by the settings

Services example

```
services:
  # Resource name
  postgres:
    # Container name, otherwise random
    container_name: postgres
    # Container image to use
    image: postgres:latest
    # Any port mapping required
    ports:
      # Network details
      - "5432:5432"
  # Next resource
  pgadmin:
    ...
```

- Resource name
- `container_name:` , the assigned name of the container otherwise it's random
- `image:` , which container image to use
- `ports:` , contains a list of any port mapping required
- Followed by next resources required

Additional comments

- `config.yaml` syntax is extensive
 - Covering very small portion of `compose.yaml` options
 - Review the documentation!
- It's typically not required to build a `compose.yaml` file from scratch

¹ <https://docs.docker.com/compose/compose-file/>

Let's practice!
INTERMEDIATE DOCKER

Dependencies and troubleshooting in Docker Compose

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

What are dependencies?

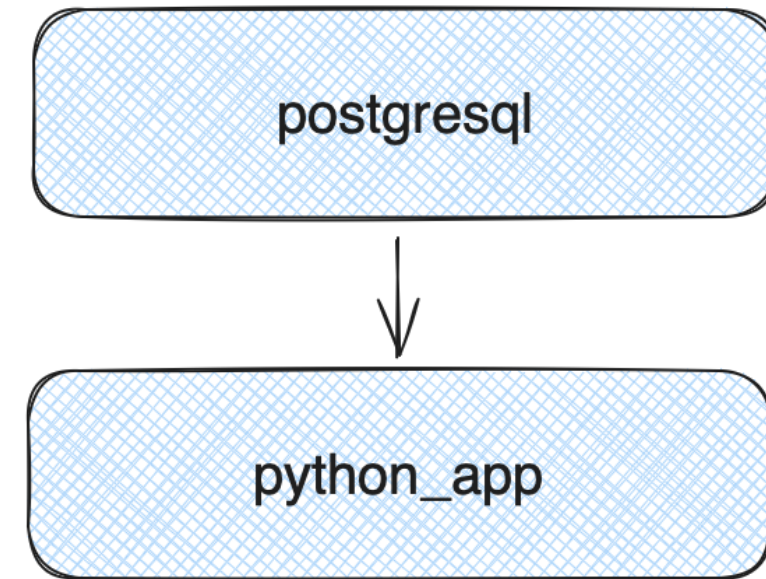
- Dependencies define the order of resource startup
- Resources (containers) may require other resources
- Example web application
 - Database container `postgresql` must start first



postgresql

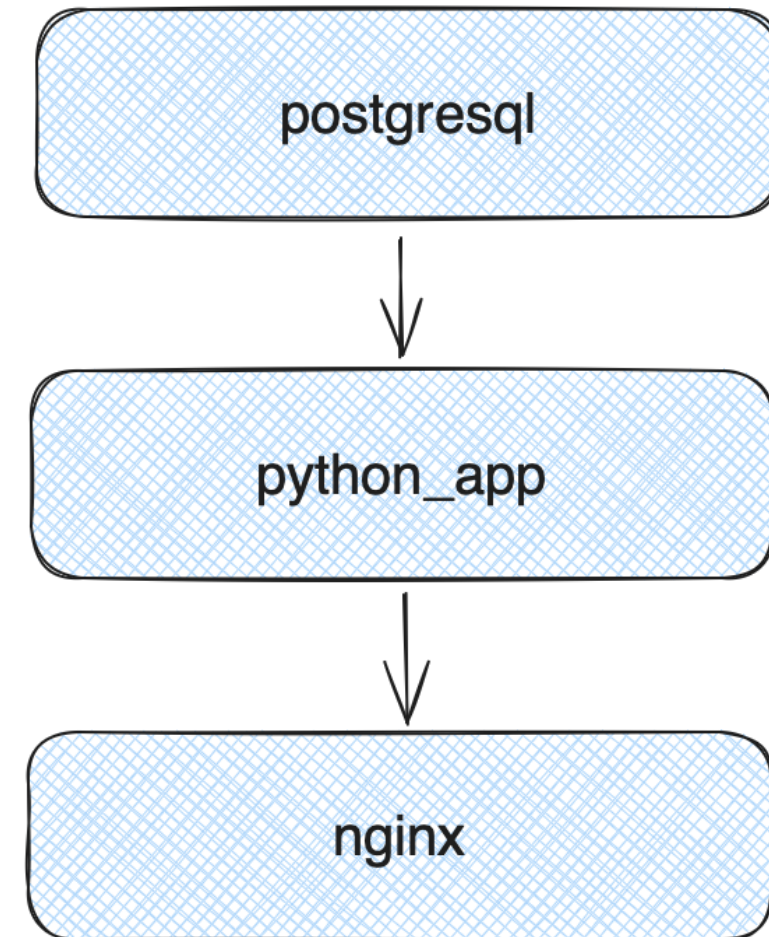
What are dependencies?

- Dependencies define the order of resources
- Resources (containers) may require other resources
- Example web application
 - Database container `postgresql` must start first
 - Then the `python_app`



What are dependencies?

- Dependencies define the order of resources
- Resources (containers) may require other resources
- Example web application
 - Database container `postgresql` must start first
 - Then the `python_app`
 - Finally, the `nginx` web server



depends_on

- Dependencies defined using the `depends_on` attribute
- Can chain dependencies as per example
- Or, can have multiple dependencies per resource if required
- Order of the `compose.yaml` file does not matter

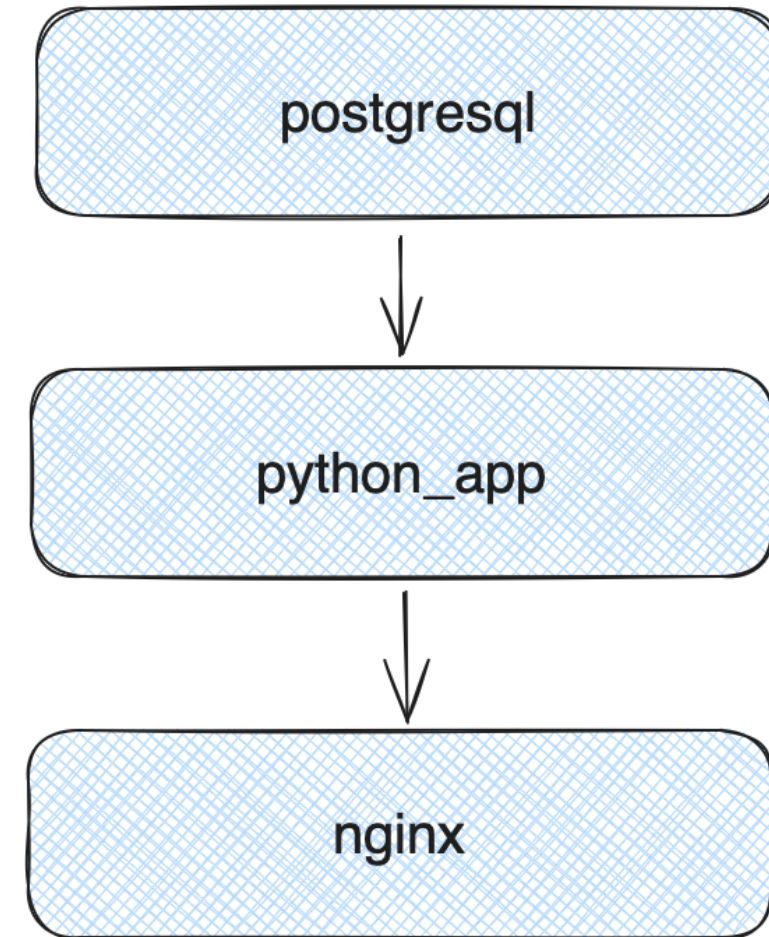
```
services:
  postgresql:
    image: postgresql:latest

  python_app:
    image: custom_app
    depends_on:
      - postgresql

  nginx:
    image: nginx/latest
    depends_on:
      - python_app
```

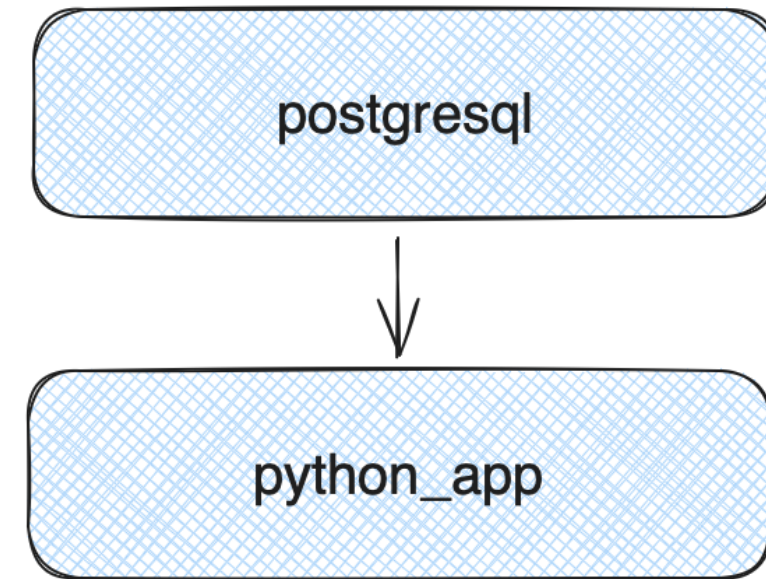
Shutting down applications

- Shutting down an application occurs in reverse order
- Stops `nginx` resource



Shutting down applications

- Shutting down an application occurs in reverse order
- Stops `nginx` resource
- Then stops the `python_app` resource



Shutting down applications

- Shutting down an application occurs in reverse order
- Stops `nginx` resource
- Then stops the `python_app` resource
- And finally the `postgresql` resource



postgresql

Other options

- Docker Compose provides other options for dependencies
- `condition:` defines how to decide when resource is ready.
 - `service_started` - Resource has started normally
 - Default behavior
 - `service_completed_successfully` - Resource ran to completion, such as a initial configuration / etc
 - `service_healthy` - Resource meets a criteria defined by `healthcheck`

```
services:
  nginx:
    image: nginx/latest
    depends_on:
      python_app:
        condition: service_started

python_app:
  image: custom_app
  depends_on:
    postgresql:
      condition: service_healthy
```

Docker Compose troubleshooting tools

- Docker Compose has additional troubleshooting tools
- `docker compose logs` - Gathers output from all resources in application

```
redis-1 | * o000o000o000o Redis is starting o000o000o000o
redis-1 | * Running mode=standalone, port=6379.
redis-1 | * Server initialized
redis-1 | * Ready to accept connections tcp
web-1   | * Serving Flask app 'app.py'
web-1   | * Running on all addresses (0.0.0.0)
web-1   | * Running on http://172.20.0.2:5000
web-1   | Press CTRL+C to quit
```

- `docker compose logs <resourcename>`

docker compose top

- `docker compose top` shows status of resources within an application

```
composetest-redis-1
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
999	2767	2726	0	01:16	?	00:03:27	redis-server *:6379

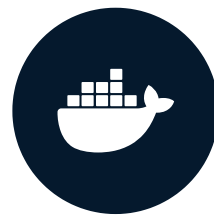
```
composetest-web-1
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	2768	2740	0	01:16	?	00:00:23	/usr/local/bin/python /usr/local/

Let's practice!
INTERMEDIATE DOCKER

Creating a data service within Docker

INTERMEDIATE DOCKER



Mike Metzger
Data Engineering Consultant

Data sharing

- `docker run -v <host directory>:<container directory>`
 - `-v ~/hostdata:/containerdata`

Data sharing in compose.yaml

- Also present in `compose.yaml` files

```
services:
  resource:
    name: resource1

# Section named volumes
volumes:
  - <host directory>:<container directory>
# Such as:
  - ~/hostdata:/containerdata
```

Networks

- `docker run --network <networkname>`
 - `docker run --network net1`
- In `compose.yaml` resources

```
services:  
  resource:  
    name: resource1  
  
  networks:  
    network_name:  
      # Such as:  
      net1:
```

Port mapping

- `docker run -p hostport:containerport`
 - `-p 8000:8000`
- Available in `compose.yaml` resources

```
services:  
  resource:  
    name: resource1  
  
  ports:  
    - hostport:containerport  
    # Such as:  
    - 8000:8000
```

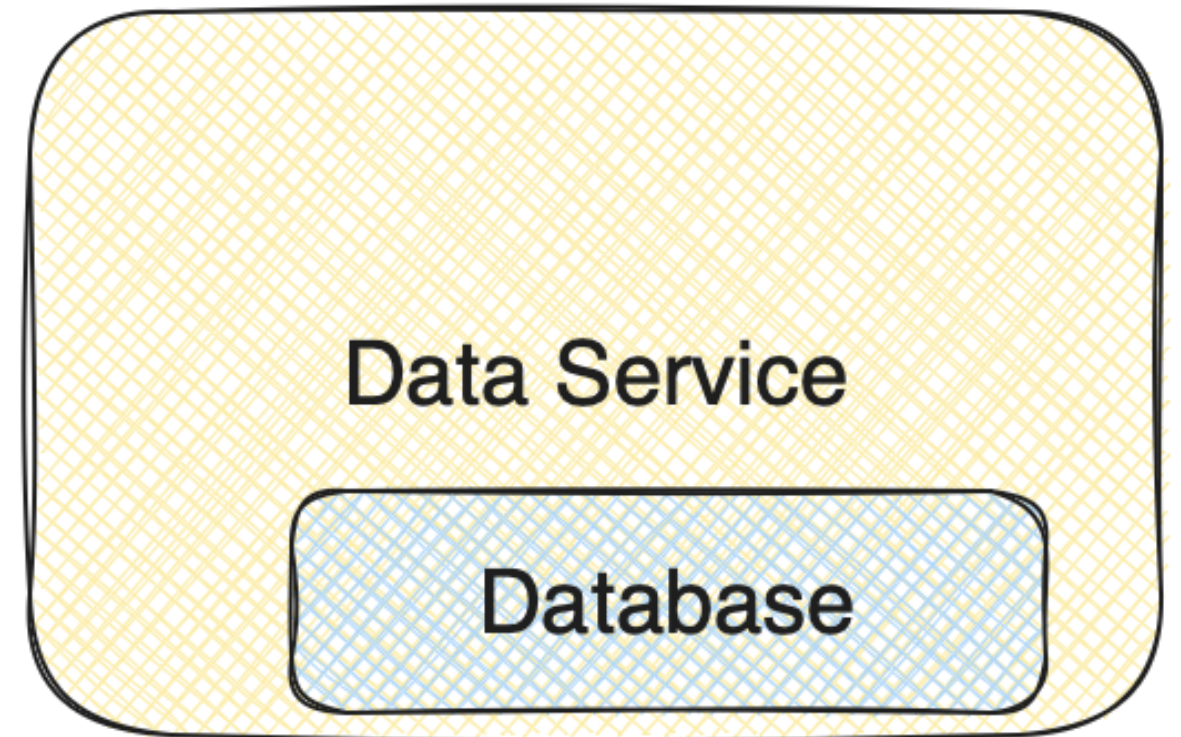

docker inspect

- Determine information about provisioned containers
 - `docker inspect <id / name>`
- Provides various levels of information
 - `Mounts` : Provides mounted data information
 - `NetworkSettings` : Network information
 - `NetworkSettings:Networks` : Shows the Docker network(s) connection details

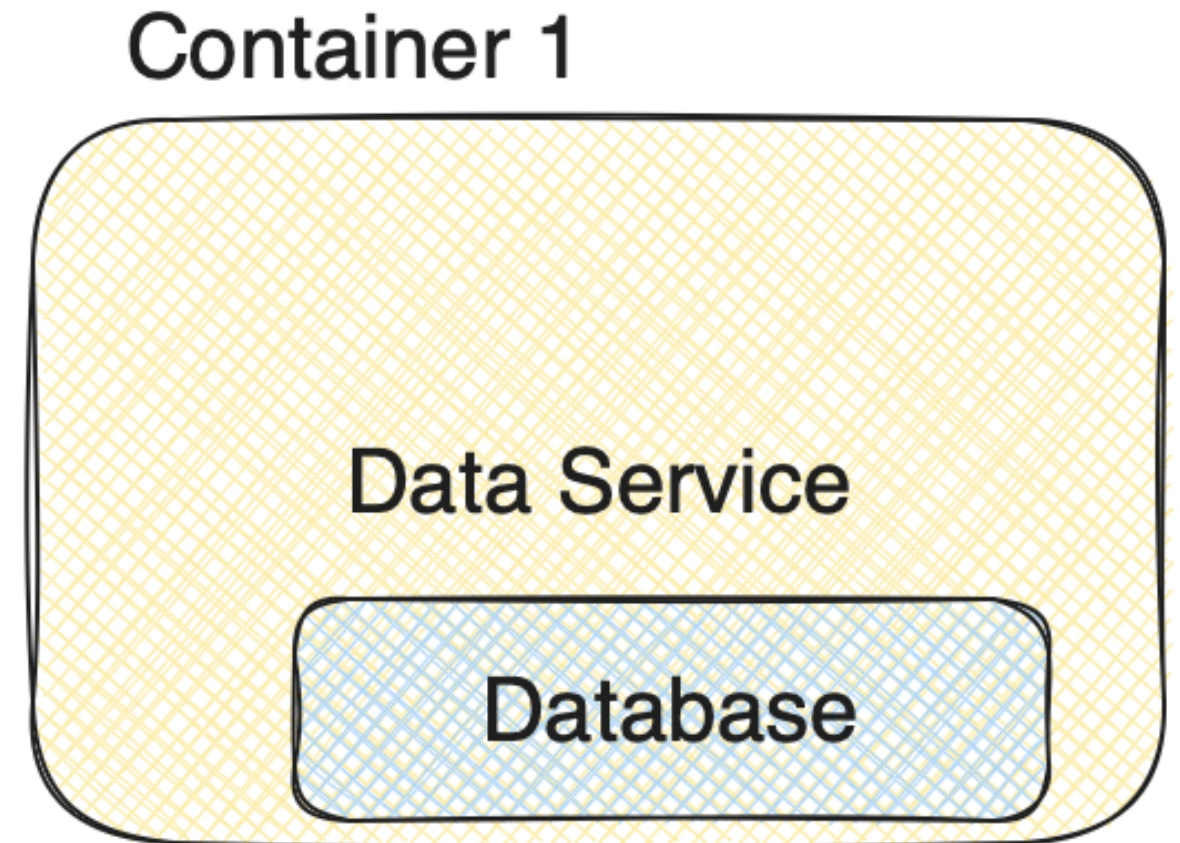
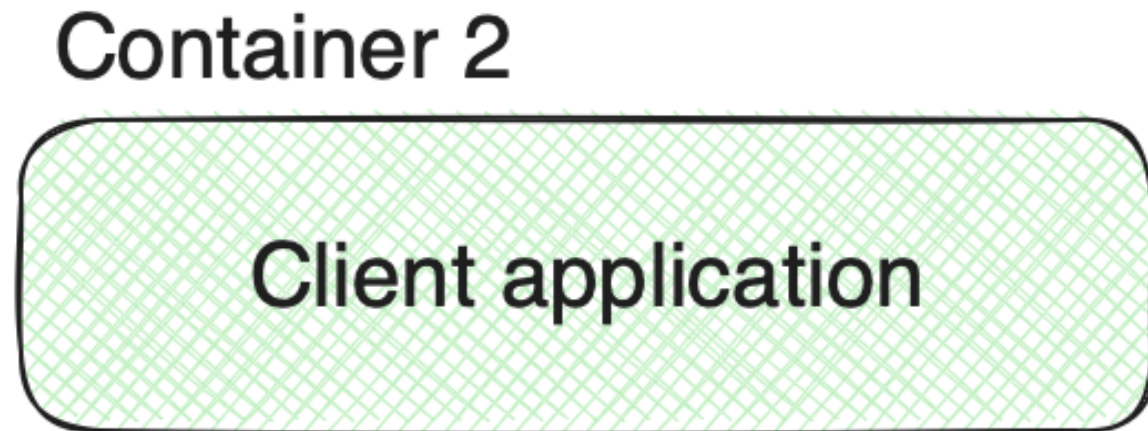
```
"Config": {  
  "Mounts": [...]  
  ...  
  "Networks": {  
    "network1": {  
      ...  
    }  
  }  
}
```


Data service

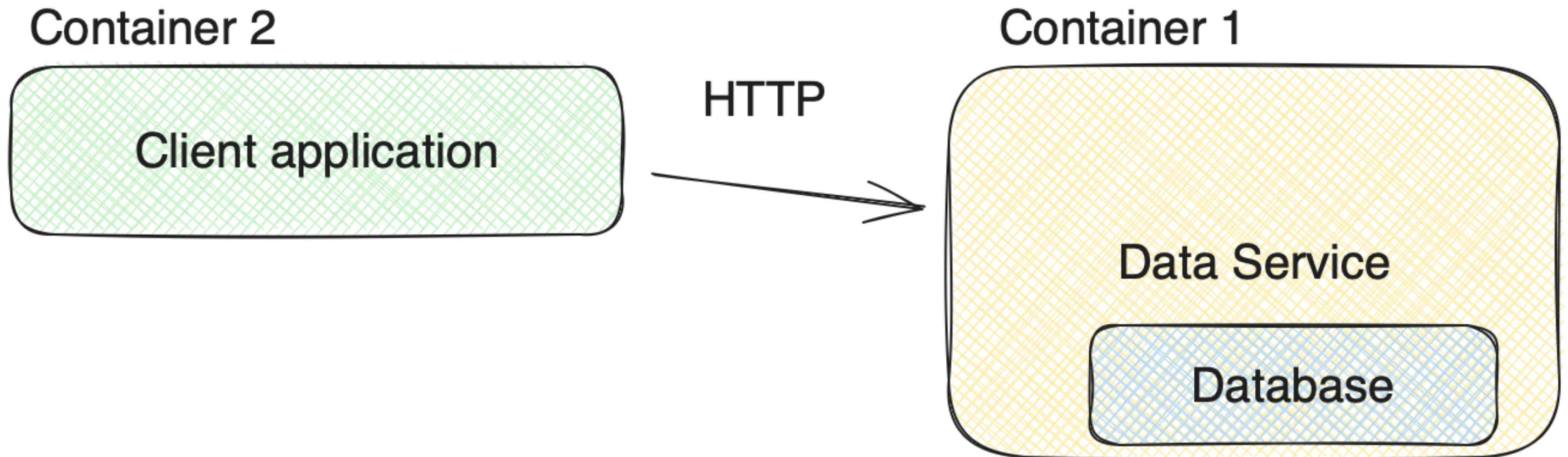
Container 1



Data service



Data service



Let's practice!
INTERMEDIATE DOCKER

Course review

INTERMEDIATE DOCKER



Mike Metzger

Data Engineering Consultant

Next steps

- Review Docker documentation docs.docker.com
- Containerize more applications
- Create custom repositories
- Docker Swarm
- Kubernetes
- CI/CD
- Mapping to host GPU hardware

Congratulations!

INTERMEDIATE DOCKER